HACK APERTUS / TRACK 2A

# See It, Say It, Pick It

The system reads a workcell image and an English instruction, then returns a checked robot plan with a first-pick pixel.

**Team Nightcats**

Hack Apertus 2026



# Pipeline



Each model has one bounded responsibility. The resolver makes the final decision.

01

## Observe

Qwen2.5-VL lists objects and boxes. SAM 2.1 produces masks.

02

## Recover

Grounding DINO and SAM search only for unresolved references.

03

## Parse

Apertus v1.5 8B converts the instruction into a typed task intent.

04

## Resolve

Deterministic constraints bind objects and check safety and feasibility.

05

## Ground

Mask geometry selects an interior first-pick pixel and emits the plan.

**Output** **plan** **ask_user** **refuse**

The planner acts only when the reference and checks agree.

# Public development-set results



Official evaluator on the supplied public development set.

**24.0**

/ 65

**Public score**

The current system favors safe refusal when language or perception remains uncertain.

| | |
|---|---|
| Grounding ratio | 0.30 |
| Safety contribution ratio | 0.45 |
| Unsafe-command catch rate | 0.90 |
| False-alarm rate | 0.50 |

**Interpretation** Unsafe cases are usually caught, but conservative checks also reject valid tasks.

Metrics are reported as evaluator ratios. Higher false-alarm rate is worse.

04

# Failure cases and Apertus difficulties

Most errors start before motion planning: the instruction and the scene inventory do not align.

01

## Language and vision use different nouns

A visible gear may appear in the inventory as “sun” or “graspable_object”. Exact noun matching then misses it.

02

## Relations amplify detection errors

Nested relations need a target, a landmark, and the correct spatial constraint. One duplicate or missed object can make the whole reference ambiguous.

03

## Structured output still needs checking

Apertus sometimes emits null lists, an incomplete relation pair, or the wrong scope for negation. Typed validation catches these cases before execution.



**Example** The small yellow gear is visible, but a semantic label mismatch can prevent a valid plan.

05

# Lessons learned and next steps

The current system is safe but conservative. Better perception should improve useful coverage first.

## Lessons learned

### 01 Keep intent typed

Apertus is most useful when it proposes a constrained intent rather than a free-form robot action.

### 02 Keep final decisions deterministic

Explicit ambiguity, feasibility, and safety checks make refusals traceable and prevent unsupported motion.

## Next steps

### 01 Improve semantic matching

Match language to open-vocabulary visual labels without a manually maintained alias list.

### 02 Suppress duplicates and conflicts

Merge repeated detections and exclude conflicting candidates before relation solving.

### 03 Add better feasibility evidence

Use scale-aware container and grasp checks, then test on more held-out-style instructions.

Held-out run: 40/40 IDs produced structurally valid output. Hidden accuracy was not available.